

JAVASCRIPT — COMPLETE STUDY NOTES

1. JAVASCRIPT BASICS

What is JavaScript?

JavaScript is a programming language used to create **dynamic and interactive web pages**.

Basic Flow

```
JavaScript Code
  ↓
Browser
  ↓
JavaScript Engine
  ↓
Memory
  ↓
Execution
  ↓
Result
```

Web Technologies

```
HTML      → Structure
CSS       → Design / Styling
JavaScript → Logic / Behaviour
```

Example project:

```
project/
├─ index.html
├─ style.css
└─ script.js
```

JavaScript can be connected to HTML using:

```
<script src="script.js"></script>
```

2. JAVASCRIPT ENGINE

A **JavaScript Engine** understands and executes JavaScript code.

Common Engines

Browser	Engine
Chrome	V8
Edge	V8
Firefox	SpiderMonkey
Safari	JavaScriptCore

V8

V8 is Google's JavaScript engine used by Chrome.

```
JavaScript Code
  ↓
V8 Engine
  ↓
Execution
  ↓
Result
```

Modern JavaScript engines use parsing, interpretation and **JIT (Just-In-Time) compilation** techniques.

3. VARIABLES

A variable is a **named reference used to store or access data**.

```
let username = "React";
```

Breakdown:

```
let
  ↓
Keyword
```

```
username
↓
Variable Name

=
↓
Assignment Operator

"React"
↓
Value
```

Declaration

Creating a variable without assigning a value:

```
let username;
```

Initialization

Giving a variable its initial value:

```
let username = "React";
```

Assignment

Giving or changing a value:

```
let username;

username = "React";
```

4. var, let, const

JavaScript has three main variable declaration keywords:

```
var
let
const
```

var

Traditional variable declaration.

```
var username = "React";

username = "JavaScript";
```

`var` can be redeclared and reassigned.

let

Modern variable declaration.

```
let city = "Chennai";

city = "Bangalore";
```

`let` can be reassigned but cannot be redeclared in the same scope.

const

Used when the variable binding should not be reassigned.

```
const course = "JavaScript";
```

This is not allowed:

```
course = "React";
```

Quick Table

Feature	var	let	const
Traditional	Yes	No	No

Feature	var	let	const
Block scoped	No	Yes	Yes
Reassign	Yes	Yes	No
Redeclare same scope	Yes	No	No
Must initialize	No	No	Yes
TDZ	No	Yes	Yes

`let` and `const` were introduced with ES6 (ECMAScript 2015).

5. VARIABLE NAMING RULES

Valid:

```
let username;
let user1;
let user_name;
let $price;
let userName;
```

Rules:

- Can contain letters
- Can contain numbers
- Cannot start with a number
- Can contain `_`
- Can contain `$`
- Cannot contain spaces
- Cannot use reserved keywords
- JavaScript is case-sensitive

Invalid:

```
let 1user;
let user name;
let var;
```

Naming Convention

JavaScript commonly uses **camelCase**:

```
let firstName;  
let lastName;  
let userEmail;  
let mobileNumber;  
let totalAmount;
```

6. ES6

ES6 = **ECMAScript 2015**

Important features include:

```
let  
const  
Arrow Functions  
Classes  
Promises  
Template Literals  
Destructuring  
Modules
```

For variables, the important ES6 keywords are:

```
let  
const
```

7. DATA TYPES

A data type describes the **kind of value** stored or represented.

Primitive Data Types

1. String
2. Number
3. BigInt
4. Boolean
5. Undefined

- 6. Null
- 7. Symbol

Object

```
Object  
Array  
Function  
Date  
Map  
Set
```

8. STRING

String represents text.

```
let name = "Rahul";  
let city = 'Chennai';  
let course = `JavaScript`;
```

Three common forms:

```
"Hello"  
'Hello'  
`Hello`
```

9. NUMBER

Number represents numeric values.

```
let age = 25;  
let price = 500;  
let percentage = 85.5;
```

JavaScript's Number type handles both:

Integer
Floating-point number

Special Number Values

NaN
Infinity
-Infinity

Example:

```
let result = "Hello" * 5;  
  
console.log(result);
```

Output:

NaN

10. BOOLEAN

Boolean has only two values:

true
false

Example:

```
let isStudent = true;  
let isAdmin = false;
```

Example:

```
let age = 20;  
  
let isAdult = age >= 18;
```

```
console.log(isAdult);
```

Output:

```
true
```

11. UNDEFINED

A variable that has been declared but has not been assigned a value has the value `undefined`.

```
let username;  
  
console.log(username);
```

Output:

```
undefined
```

Memory Trick

```
undefined  
↓  
No value assigned yet
```

12. NULL

`null` represents an **intentional absence of a value**.

```
let selectedUser = null;
```

undefined vs null

undefined → No value assigned yet

null → Intentionally empty

Important JavaScript quirk:

```
typeof null
```

returns:

```
"object"
```

13. BIGINT

BigInt is used for very large integers beyond the safe integer range of Number.

```
let bigNumber = 123456789012345678901234567890n;
```

The `n` indicates a BigInt literal.

14. SYMBOL

Symbol creates unique values.

```
let id1 = Symbol("id");  
let id2 = Symbol("id");  
  
console.log(id1 === id2);
```

Output:

```
false
```

15. OBJECT

Objects store related data as **key-value pairs**.

```
let student = {  
  name: "Rahul",  
  age: 22,  
  course: "JavaScript"  
};
```

Here:

```
name   → Property  
Rahul  → Value  
  
age    → Property  
22     → Value
```

Access:

```
student.name;  
student.age;
```

16. ARRAY

An array stores multiple values in an ordered collection.

```
let fruits = [  
  "Apple",  
  "Mango",  
  "Orange",  
  "Banana"  
];
```

Array Index

Array indexing starts from **0**.

```
console.log(fruits[0]);
```

Output:

```
Apple
```

Index:	0	1	2	3
	↓	↓	↓	↓
	Apple	Mango	Orange	Banana

Last element:

```
fruits[fruits.length - 1];
```

17. FUNCTION

A function is a **reusable block of code**.

```
function greet() {  
    console.log("Hello");  
}
```

Call:

```
greet();
```

18. typeof OPERATOR

`typeof` is used to check the type of a value.

```
typeof "React"
```

Output:

```
"string"
```

Examples:

```
typeof 100;           // "number"  
typeof true;          // "boolean"  
typeof undefined;     // "undefined"  
typeof 100n;           // "bigint"  
typeof Symbol("id");  // "symbol"  
typeof {};             // "object"  
typeof [];             // "object"  
typeof null;           // "object"
```

19. DYNAMIC TYPING

JavaScript is **dynamically typed**.

A variable can refer to values of different types during execution.

```
let value = "React";  
  
value = 100;  
  
value = true;
```

The same variable can hold different types at different times.

20. PRIMITIVE vs OBJECT

Primitive

```
let a = 10;  
let b = a;  
  
b = 20;  
  
console.log(a);
```

Output:

10

The original primitive value is unchanged.

Object Reference

```
let student1 = {  
  name: "Rahul"  
};  
  
let student2 = student1;  
  
student2.name = "Guna";  
  
console.log(student1.name);
```

Output:

Guna

Both variables refer to the same object.

21. STACK AND HEAP — BASIC IDEA

A simplified mental model:

```
Primitive / local execution data  
    ↓  
    Stack  
  
Objects / dynamically allocated data  
    ↓  
    Heap
```

This is a simplified model; JavaScript specifications do not require a particular stack/heap implementation.

22. OPERATORS

Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus / Remainder
**	Exponentiation

Example:

```
let a = 10;
let b = 3;

console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
console.log(a ** b); // 1000
```

Assignment Operators

```
=      Assignment
+=     Add and assign
-=     Subtract and assign
*=     Multiply and assign
/=     Divide and assign
%=     Modulus and assign
**=    Exponentiation and assign
```

Example:

```
let x = 10;
```

```
x += 5;  
  
console.log(x);
```

Output:

15

Comparison Operators

==	Equal
===	Strict Equal
!=	Not Equal
!==	Strict Not Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Logical Operators

&&	AND
	OR
!	NOT

Increment / Decrement

```
i++;  
i--;
```

`++` → increase by 1

`--` → decrease by 1

23. CONDITIONS

A condition checks whether something is **true or false**.

```
let age = 20;  
  
age >= 18;
```

Result:

```
true
```

Conditions are used to make decisions in programs.

24. if

Used when code should execute only when a condition is true.

```
if (condition) {  
    // code  
}
```

Example:

```
let age = 20;  
  
if (age >= 18) {  
    console.log("Eligible");  
}
```

25. if...else

Used when there are two possible results.

```
if (condition) {  
    // true  
} else {  
    // false  
}
```

Example:

```
let age = 16;

if (age >= 18) {
  console.log("Eligible");
} else {
  console.log("Not Eligible");
}
```

26. if...else if...else

Used when there are multiple conditions.

```
if (condition1) {

} else if (condition2) {

} else {

}
```

Example:

```
let mark = 75;

if (mark >= 90) {
  console.log("A+");
} else if (mark >= 75) {
  console.log("A");
} else if (mark >= 50) {
  console.log("B");
} else {
  console.log("Fail");
}
```

27. NESTED if

An `if` inside another `if`.

```
let age = 22;
let hasId = true;

if (age >= 18) {

  if (hasId) {
    console.log("Entry Allowed");
  }

}
```

28. TERNARY OPERATOR

Short form of simple `if...else`.

Syntax

```
condition ? trueValue : falseValue;
```

Example:

```
let age = 20;

let result = age >= 18
  ? "Adult"
  : "Minor";

console.log(result);
```

29. SWITCH

`switch` is useful when comparing **one value against multiple fixed values**.

Syntax

```
switch (expression) {

  case value1:
```

```
    // code
    break;

case value2:
    // code
    break;

default:
    // code
}
```

Example:

```
let day = 2;

switch (day) {

    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
        break;

    case 3:
        console.log("Wednesday");
        break;

    default:
        console.log("Invalid Day");
}
```

Output:

Tuesday

Important

`break` stops the switch.

Without `break`, execution can continue into the following cases. This is called **fall-through**.

if vs switch

```
if
↓
Conditions / ranges / complex expressions

switch
↓
One value + multiple fixed cases
```

30. CONTROL STATEMENTS

Control statements control the **flow of program execution**.

Types

```
CONTROL STATEMENTS
|
├── Conditional
|   ├── if
|   ├── if...else
|   ├── else if
|   └── switch
|
├── Looping
|   ├── for
|   ├── while
|   └── do...while
|
└── Jump / Transfer
    ├── break
    ├── continue
    └── return
```

31. LOOPS

A loop repeatedly executes code.

Why use loops?

- Avoid repeated code
- Process multiple values
- Work with arrays
- Automate repeated tasks

Basic loops:

```
for  
while  
do...while
```

32. for LOOP

Used when the number of iterations is generally known.

Syntax

```
for (initialization; condition; update) {  
    // code  
}
```

Example:

```
for (let i = 1; i <= 5; i++) {  
    console.log(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

Flow

```
Initialization
  ↓
Condition
  ↓
  True?
  ↓
Execute
  ↓
Update
  ↓
Condition Again
  ↓
  False
  ↓
Exit
```

33. while LOOP

Used to execute code while a condition is true.

Syntax

```
while (condition) {
    // code
}
```

Example:

```
let i = 1;

while (i <= 5) {

    console.log(i);

    i++;
}
```

Important

Always make sure the condition eventually becomes false.

Otherwise:

Infinite Loop

can occur.

34. do...while

The `do...while` loop executes the code **at least once** before checking the condition.

Syntax

```
do {  
    // code  
} while (condition);
```

Example:

```
let i = 1;  
  
do {  
    console.log(i);  
    i++;  
} while (i <= 5);
```

Main Difference

```
for  
↓  
Initialization → Condition → Code → Update  
  
while  
↓  
Initialization → Condition → Code → Update
```

```
do...while
↓
Code → Update → Condition
```

Key Point

```
let i = 10;

do {
  console.log(i);
} while (i <= 5);
```

Output:

```
10
```

Why?

Because the code executes before the condition is checked.

35. break

`break` immediately stops a loop.

```
for (let i = 1; i <= 10; i++) {

  if (i === 5) {
    break;
  }

  console.log(i);
}
```

Output:

```
1
2
```

```
3  
4
```

Memory:

```
break → STOP
```

36. continue

`continue` skips the current iteration and moves to the next one.

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue;  
  }  
  console.log(i);  
}
```

Output:

```
1  
2  
4  
5
```

Memory:

```
continue → SKIP CURRENT ITERATION
```

37. LOOP LOGIC PATTERNS

Pattern 1 — Print

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

Pattern 2 — Collect

```
let result = "";  
  
for (let i = 1; i <= 5; i++) {  
  result = result + i + " ";  
}  
  
console.log(result);
```

Output:

```
1 2 3 4 5
```

Pattern 3 — Calculate

```
let sum = 0;  
  
for (let i = 1; i <= 5; i++) {  
  sum = sum + i;  
}  
  
console.log(sum);
```

Output:

```
15
```

Pattern 4 — Find

```
for (let i = 1; i <= 10; i++) {  
  
    if (i === 7) {  
        console.log(i);  
        break;  
    }  
  
}
```

38. SINGLE OUTPUT vs SINGLE LINE

Single Output

One final answer:

15

Example:

```
let sum = 0;  
  
for (let i = 1; i <= 5; i++) {  
    sum = sum + i;  
}  
  
console.log(sum);
```

Single Line

Multiple values on one line:

1 2 3 4 5

Example:

```
let result = "";
```

```
for (let i = 1; i <= 5; i++) {  
    result = result + i + " ";  
}  
  
console.log(result);
```

Memory:

Single Output → One final answer

Single Line → Multiple values in one line

Multi Line → Each value on separate line

39. ARRAYS

An array stores multiple values in one variable.

```
let fruits = [  
    "Apple",  
    "Mango",  
    "Orange",  
    "Banana"  
];
```

Index starts at `0`.

```
fruits[0]; // Apple  
fruits[1]; // Mango  
fruits[2]; // Orange
```

Array + for

```
let fruits = ["Apple", "Mango", "Orange"];  
  
for (let i = 0; i < fruits.length; i++) {  
    console.log(fruits[i]);  
}
```

Important:

```
fruits[i]
```

means:

Access the current array value.

40. COMMON ARRAY METHODS

push()

Adds to the end.

```
let fruits = ["Apple", "Mango"];  
fruits.push("Orange");
```

pop()

Removes the last element.

```
fruits.pop();
```

unshift()

Adds to the beginning.

```
fruits.unshift("Banana");
```

shift()

Removes the first element.

```
fruits.shift();
```

concat()

Combines arrays.

```
let a = [1, 2];
let b = [3, 4];

let result = a.concat(b);
```

Result:

```
[1, 2, 3, 4]
```

41. OBJECTS

Objects store related information using key-value pairs.

```
let student = {
  name: "Rahul",
  age: 22,
  course: "JavaScript"
};
```

Access:

```
student.name;
student.age;
student.course;
```

Bracket notation:

```
student["name"];
```

42. FUNCTIONS

A function is reusable code.

```
function greet() {
  console.log("Hello");
}
```

```
greet();
```

Function with Parameters

```
function add(a, b) {  
  console.log(a + b);  
}  
  
add(10, 20);
```

return

```
function add(a, b) {  
  return a + b;  
}  
  
let result = add(10, 20);  
  
console.log(result);
```

43. ARROW FUNCTION

Basic syntax:

```
const greet = () => {  
  console.log("Hello");  
};
```

With parameters:

```
const add = (a, b) => {  
  return a + b;  
};
```

Short form:

```
const add = (a, b) => a + b;
```

44. CALLBACK FUNCTION

A callback is a function passed as an argument to another function and executed later.

Basic Syntax

```
const mainFunction = (callback) => {  
  callback();  
};
```

Example:

```
const greet = () => {  
  console.log("Hello");  
};  
  
const processUser = (callback) => {  
  console.log("Processing...");  
  callback();  
};  
  
processUser(greet);
```

Output:

```
Processing...  
Hello
```

Callbacks are common with:

```
Events  
Timers  
Asynchronous operations
```

Too many nested callbacks can create **Callback Hell**.

45. setTimeout()

Used to execute a function after a delay.

```
const showMessage = () => {  
  console.log("Data Loaded");  
};  
  
setTimeout(showMessage, 2000);
```

Flow:

```
setTimeout()  
  ↓  
Wait 2 seconds  
  ↓  
Callback executes  
  ↓  
Data Loaded
```

46. PROMISE

A Promise represents the future result of an asynchronous operation.

Promise States

```
Pending  
  ↓  
├── Fulfilled  
└── Rejected
```

Meaning

```
Pending    → Still running  
  
Fulfilled  → Success  
  
Rejected   → Failure
```

Syntax

```
const promise = new Promise((resolve, reject) => {  
  
    // asynchronous operation  
  
});
```

```
resolve() → Success  
reject()  → Failure
```

Example:

```
const promise = new Promise((resolve, reject) => {  
  
    const success = true;  
  
    if (success) {  
        resolve("Task Completed");  
    } else {  
        reject("Task Failed");  
    }  
  
});
```

47. .then()

`.then()` handles a successfully resolved Promise.

```
promise.then((result) => {  
    console.log(result);  
});
```

48. .catch()

`.catch()` handles a rejected Promise or error.

```
promise
  .then((result) => {
    console.log(result);
  })
  .catch((error) => {
    console.log(error);
  });
```

49. .finally()

`.finally()` runs after the Promise settles, whether it succeeds or fails.

```
promise
  .then((result) => {
    console.log(result);
  })
  .catch((error) => {
    console.log(error);
  })
  .finally(() => {
    console.log("Finished");
  });
```

Memory:

```
then()    → Success
catch()   → Failure
finally() → Always runs after settlement
```

50. ASYNCHRONOUS JAVASCRIPT

Some operations take time:

```
API calls
Network requests
Timers
```

File loading
Database requests

Instead of blocking the application, JavaScript can continue other work while the operation is being completed.

Basic idea:

```
Start Task
  ↓
Task Takes Time
  ↓
JavaScript Continues Other Work
  ↓
Task Completes
  ↓
Handle Result
```

Important async concepts:

Callback
Promise
async
await
fetch
.then()
.catch()

51. async

`async` marks a function as asynchronous.

An `async` function always returns a Promise.

```
const getData = async () => {  
  
};
```

52. await

`await` waits for a Promise inside an async function.

```
const getData = async () => {  
    const result = await promise;  
};
```

Memory:

```
async → Function works with asynchronous code  
await → Wait for Promise result
```

53. try...catch

Used to handle errors.

```
try {  
    // code that may fail  
} catch (error) {  
    // handle error  
}
```

With async/await:

```
const getData = async () => {  
    try {  
        const result = await promise;  
    } catch (error) {
```

```
        console.log(error);  
    }  
};
```

54. CALLBACK vs PROMISE vs ASYNC/AWAIT

```
Callback  
  ↓  
Nested Callback Problem  
  ↓  
Promise  
  ↓  
.then() / .catch()  
  ↓  
Async/Await  
  ↓  
try/catch
```

Callback

```
Pass function  
Execute later
```

Problem:

```
Callback Hell
```

Promise

```
resolve → Success  
reject  → Failure
```

Async/Await

Cleaner syntax
Easier to read
try/catch for errors

Both `.then()` and `await` work with Promises.

55. FETCH API

Fetch API is a JavaScript Web API used to communicate with servers through HTTP requests.

Basic syntax:

```
fetch("API_URL");
```

Flow:

```
Frontend
  ↓
fetch()
  ↓
HTTP Request
  ↓
Backend / API
  ↓
HTTP Response
  ↓
JavaScript
```

Common real-world uses:

Login
Registration
Products
Dashboard
Employee details
Orders

56. fetch() + .then()

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then((response) => response.json())
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.log(error);
  });
```

57. response.json()

`fetch()` gives a **Response object**.

To read JSON data:

```
response.json()
```

Important:

`response.json()` also returns a Promise.

Flow:

```
fetch()
  ↓
Response Object
  ↓
response.json()
  ↓
JavaScript Data
```

58. fetch() + async/await

```
const getUsers = async () => {
```

```
    try {  
        const response = await fetch(  
            "https://jsonplaceholder.typicode.com/users"  
        );  
        const data = await response.json();  
        console.log(data);  
    } catch (error) {  
        console.log(error);  
    }  
};  
getUsers();
```

Why two awaits?

First:

```
await fetch(url);
```

Waits for the HTTP response.

Second:

```
await response.json();
```

Waits for the response body to be converted into JavaScript data.

```
API  
↓  
await fetch()  
↓  
Response  
↓  
await response.json()
```

↓
Data

59. response.ok

Very important for interviews:

`fetch()` does not automatically reject just because the server returns HTTP statuses such as `404` or `500`.

So check:

```
if (!response.ok) {  
  throw new Error("Request Failed");  
}
```

Complete pattern:

```
const getData = async () => {  
  
  try {  
  
    const response = await fetch(url);  
  
    if (!response.ok) {  
      throw new Error("Failed to fetch data");  
    }  
  
    const data = await response.json();  
  
    console.log(data);  
  
  } catch (error) {  
  
    console.log(error.message);  
  
  }  
  
};
```

60. HTTP METHODS

Method	Purpose
GET	Read data
POST	Create/send data
PUT	Update/replace data
DELETE	Delete data

61. GET

Used to retrieve data.

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => console.log(error));
```

62. POST

Used to send/create new data.

```
const user = {
  name: "Arun",
  email: "arun@gmail.com"
};

fetch("https://jsonplaceholder.typicode.com/users", {

  method: "POST",

  headers: {
    "Content-Type": "application/json"
  },

  body: JSON.stringify(user)

})
```

```
.then((response) => response.json())
.then((data) => {
  console.log(data);
})
.catch((error) => {
  console.log(error);
});
```

63. PUT

Used to update/replace an existing resource.

```
fetch("https://jsonplaceholder.typicode.com/users/1", {

  method: "PUT",

  headers: {
    "Content-Type": "application/json"
  },

  body: JSON.stringify({
    name: "Updated User",
    email: "updated@gmail.com"
  })

});
```

64. DELETE

Used to delete data.

```
fetch("https://jsonplaceholder.typicode.com/users/1", {

  method: "DELETE"

})
.then(() => {
  console.log("Deleted");
})
.catch((error) => {
```

```
    console.log(error);  
  });
```

65. JSON.stringify()

Converts a JavaScript object into a JSON string.

```
const user = {  
  name: "Arun",  
  email: "arun@gmail.com"  
};  
  
const data = JSON.stringify(user);
```

Flow:

```
JavaScript Object  
  ↓  
JSON.stringify()  
  ↓  
JSON String  
  ↓  
Request Body  
  ↓  
Backend
```

66. JSON.parse()

Converts JSON text into JavaScript data.

Memory:

```
SAVE / SEND  
  ↓  
JSON.stringify()  
  
READ JSON
```

↓
JSON.parse()

67. HEADERS

Headers provide additional information about an HTTP request.

Example:

```
headers: {  
  "Content-Type": "application/json"  
}
```

This tells the server that the request body contains JSON.

68. BODY

The `body` contains data being sent to the server.

```
body: JSON.stringify(user)
```

Flow:

```
Frontend Object  
  ↓  
JSON.stringify()  
  ↓  
body  
  ↓  
Backend
```

69. FETCH + DOM

Example:

```

const button = document.getElementById("loadBtn");
const output = document.getElementById("output");

button.addEventListener("click", async () => {

    try {

        const response = await fetch(
            "https://jsonplaceholder.typicode.com/users"
        );

        const users = await response.json();

        output.innerHTML = "";

        users.forEach((user) => {

            output.innerHTML += `
                <div>
                    <h3>${user.name}</h3>
                    <p>${user.email}</p>
                </div>
            `;

        });

    } catch (error) {

        output.textContent = "Failed to load users";

    }

});

```

This connects:

```

Button Click
  ↓
fetch()
  ↓
API
  ↓
JSON
  ↓
JavaScript Array
  ↓

```

```
forEach()  
  ↓  
DOM  
  ↓  
Webpage
```

70. IMPORTANT INTERVIEW FORMULA

Callback

Function passed to another function

Promise

Future asynchronous result

Promise States

Pending
Fulfilled
Rejected

Promise Methods

.then() → Success
.catch() → Error
.finally() → After settlement

async

Makes function asynchronous

await

Waits for Promise result

fetch()

Makes HTTP request

response.json()

Converts JSON response into JavaScript data

HTTP

GET → Read
POST → Create
PUT → Update
DELETE → Delete

71. IMPORTANT JAVASCRIPT MEMORY SHEET

VARIABLES

var
let
const

DATA TYPES

String
Number
Boolean
Undefined
Null
BigInt
Symbol
Object

OPERATORS

Arithmetic
Assignment
Comparison

Logical
Increment
Decrement

CONDITIONS

if
if...else
else if
Nested if
Ternary
switch

LOOPS

for
while
do...while

JUMP

break
continue
return

ASYNC

Callback
Promise
.then()
.catch()
.finally()
async
await
try
catch

FETCH

GET
POST

PUT
DELETE

72. MOST IMPORTANT LOGIC PATTERNS

Sum

```
let sum = 0;

for (let i = 1; i <= 5; i++) {
  sum = sum + i;
}

console.log(sum);
```

Count

```
let count = 0;

for (let i = 1; i <= 10; i++) {

  if (i % 2 === 0) {
    count++;
  }

}

console.log(count);
```

Even Number

```
if (num % 2 === 0) {
  console.log("Even");
}
```

Odd Number

```
if (num % 2 !== 0) {  
    console.log("Odd");  
}
```

Highest Number Pattern

```
let highest = 0;  
  
for (let i = 1; i <= 10; i++) {  
    if (i > highest) {  
        highest = i;  
    }  
}  
  
console.log(highest);
```

Array Traversal

```
let fruits = ["Apple", "Mango", "Orange"];  
  
for (let i = 0; i < fruits.length; i++) {  
    console.log(fruits[i]);  
}
```

73. GOLDEN FLOWS TO MEMORIZE

JavaScript Execution

```
Code  
↓  
Browser  
↓  
JavaScript Engine  
↓  
Memory
```

```
↓  
Execution  
↓  
Output
```

Variable

```
Keyword  
↓  
Variable Name  
↓  
Assignment Operator  
↓  
Value  
↓  
Data Type
```

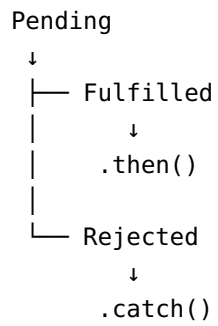
Condition

```
Condition  
↓  
true?  
├─ YES → Execute  
└─ NO  → Skip / Else
```

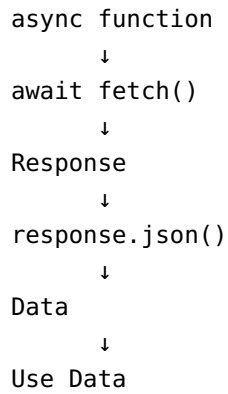
for Loop

```
Initialization  
↓  
Condition  
↓  
Execute  
↓  
Update  
↓  
Condition Again  
↓  
False → Exit
```

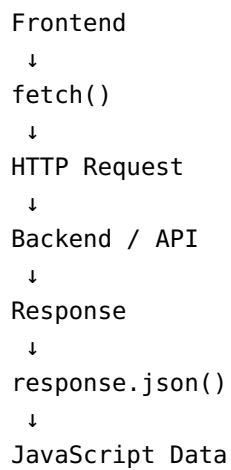
Promise



Async API



Fetch



74. TOP INTERVIEW QUESTIONS

Basics

1. What is JavaScript?
2. What is a JavaScript engine?
3. What is V8?
4. How does JavaScript execute in a browser?
5. What is ES6?

Variables

1. What is a variable?
2. What is declaration?
3. What is initialization?
4. What is assignment?
5. Difference between `var`, `let`, and `const`?
6. What is block scope?
7. What is function scope?
8. Can `let` be redeclared?
9. Can `const` be reassigned?

Data Types

1. What are primitive data types?
2. What is String?
3. What is Number?
4. What is Boolean?
5. What is undefined?
6. What is null?
7. Difference between null and undefined?
8. What is BigInt?
9. What is Symbol?
10. What is an Object?
11. What is an Array?
12. What does `typeof` do?
13. What does `typeof null` return?
14. Is JavaScript dynamically typed?

Control Flow

1. What is a condition?

2. Difference between `if` and `switch`?
3. What is a ternary operator?
4. What is a loop?
5. Difference between `for`, `while`, and `do...while`?
6. What is `break`?
7. What is `continue`?
8. What is an infinite loop?

Async JavaScript

1. What is a callback?
2. What is callback hell?
3. What is a Promise?
4. What are Promise states?
5. What does `resolve()` do?
6. What does `reject()` do?
7. What does `.then()` do?
8. What does `.catch()` do?
9. What does `.finally()` do?
10. What is `async`?
11. What is `await`?
12. Why use `try...catch`?

Fetch/API

1. What is Fetch API?
2. What does `fetch()` return?
3. What is `response.json()`?
4. Why do we use `response.ok`?
5. What is GET?
6. What is POST?
7. What is PUT?
8. What is DELETE?
9. What is `JSON.stringify()`?
10. What is a request body?
11. What are headers?

75. FINAL QUICK REVISION

JavaScript

↓

Variables

↓

Data Types

↓

Operators
↓
Conditions
↓
Loops
↓
Arrays
↓
Objects
↓
Functions
↓
Callbacks
↓
Promises
↓
Async/Await
↓
Fetch API
↓
DOM
↓
Real Applications

One-Line Memory

var/let/const
→ Variables

String/Number/Boolean/...

→ Data Types

+ - * / % **

→ Arithmetic

== === != !== > < >= <=

→ Comparison

&& || !

→ Logical

if / else / switch

→ Decisions

for / while / do...while

→ Repetition

`break / continue`

→ Loop Control

`function / arrow function`

→ Reusable Code

`callback`

→ Function passed to another function

`Promise`

→ Future async result

`async / await`

→ Cleaner async code

`fetch()`

→ HTTP/API communication

`GET / POST / PUT / DELETE`

→ CRUD-style HTTP operations

The 10 Things You Should Know First

1. `var`, `let`, `const`
2. Data types
3. Operators
4. `if / else`
5. `switch`
6. `for / while / do...while`
7. Arrays + loops
8. Functions + callbacks
9. Promises + `async/await`
10. Fetch API + JSON